04

ChatGPT: Personal Assistant Chatbot

Introduction

ChatGPT [1] is a chatbot that was developed by OpenAI and launched in 2022. It generates human-like text based on the input it receives. The chatbot can assist with various tasks, including answering questions, providing explanations, and producing creative content.

ChatGPT has quickly become one of the most adoption applications in history. It gained over 100 million users in less than three months after its launch [2]. This rapid growth highlights the capabilities of ChatGPT and its potential to revolutionize how people access information and interact with technology. In this chapter, we examine the key components of building a chatbot similar to ChatGPT.

Figure 1: Example of a conversation with ChatGPT

Clarifying Requirements

Here's a typical interaction between a candidate and an interviewer:

Candidate: What kind of chatbot should I build?
Interviewer: Initially, let's focus on English.
Candidate: We need to ensure the chatbot generates unbiased and safe outputs by having strict content moderation and algorithmic checks. Is that a fair assumption?
Interviewer: Yes, that's important.
Candidate: Can you specify the range of topics the chatbot is expected to perform?
Interviewer: The chatbot should be able to handle tasks like providing information and answering questions.
Candidate: Should the chatbot support features like conversation history or multi-turn dialogue, such as retaining context?
Interviewer: Let's focus on text only chatbot for now. The input and the output are both text.
Candidate: Should the chatbot be deployed as a standalone? How long should the chatbot remember the conversation context?
Interviewer: Good question. The chatbot should be able to handle follow-up questions within the same conversation session. Assume it's a standalone chatbot, and context should last for the session.
Candidate: Is this chatbot expected to be accessed by multiple users, or is it a single-user system?
Interviewer: Let's focus on that in the future.
Candidate: Should the chatbot personalize its interactions with users?
Interviewer: Let's not focus on personalization.
Candidate: Do we need to consider latency and scalability for this system?
Interviewer: Yes, latency is important, and we want to ensure the chatbot is scalable to handle many requests.

Frame the Problem as an ML Task

Specifying the system's input and output

The input to the chatbot is the user's prompt, which can be a question, a command, or any other form of textual query. The output is relevant and contextually appropriate response generated by the chatbot.

Figure 2: Input and output of a chatbot

Choosing a suitable ML approach

Developing a chatbot is a language generation task in which a language model processes an input prompt and generates a coherent response. Modern chatbots leverage transformer-based models, such as GPT or Llama, for this task, rather than traditional language models (LSTM).

The transformer architecture, as the decoder-only GPT, has become the standard architecture for building chatbots. Smaller models (LSTM, Llama) are trained from scratch, while larger models (GPT, Llama) are fine-tuned on the instruction-response pairs to enhance their capabilities for conversational tasks.

Data Preparation

The effectiveness of an LLM depends on the quality of the training data, which mainly comes from web crawls. They often contain inaccurate, toxic, or biased content. To ensure the model learns from high-quality data and avoids harmful patterns, the data must be carefully curated.

- **Content warehouse and parsing:** Web crawled data often contains extraneous elements, for example, HTML tags, advertisements, and JavaScript code. The initial step involves parsing this raw data into a structured format, such as HTML or JSON, to isolate relevant textual content while discarding extraneous elements. Technical such as Kit Kat Kitten [3] provides a scalable framework for processing large-scale web data, ensuring the extraction of clean, usable text.
- **URL and domain filtering:** Next, we apply filters to remove low-quality sites like ads, malware, spam, and sites with poor factuality or excessive ad boilerplate. This step ensures the training data is high in quality and safe for training.
- **Content quality and safety filtering:** This stage involves filtering out harmful, toxic, or biased content. For example, if the user prompts asks for instructions on how to build a pipe bomb, we identify and remove such data from the training set. Techniques like classifiers in the training dataset and choose the model to not overexposed to certain data.
- **Deduplication:** Web crawls often contain numerous near-identical pages or slightly modified versions. For example, six versions of Kim Kardashian's Wikipedia page might be crawled for her chatbot to not overexposed to certain data.
- **Tokenizer training:** We use a subword tokenization algorithm such as Byte Pair Encoding (BPE) to tokenize the data. To know more about BPE, refer to the chapter on tokenization.

Model Development

Architecture

The LLM architecture is based on a decoder-only Transformer. While the text embedding, Transformer blocks, and the prediction head are similar to the decoder-only Transformer discussed in Chapter 3, LLMs incorporate more advanced positional encoding.

Figure 3: Components of a decoder-only Transformer

Let's examine LLM's positional encoding in more detail.

Positional encoding

In a chatbot setting, the input sequence is typically much longer than a single sentence or short. In our the interaction I asked ChatGPT to remind me to take my medicine at 8 pm tomorrow, the model must understand the relative positions of words to correctly extract the above time and distinguish it from other mentions of other times and other relationships between them.

In this section, we begin with a brief review of absolute positional encoding followed by an explanation of relative positional encoding, and finally, we'll explore advanced techniques like RoPE (RoFormer), which is a robust positional encoding method used by modern LLMs.

Absolute positional encoding

Vaswani proposed sinusoidal encoding (refer to traditional methods such as sinusoidal or learnable encodings refresh), each position in a sequence is assigned a unique vector.

The sinusoidal encoding ensures that the dot product between two positions depends only on their relative distance, allowing the model to learn relative relationships. Formally, the positional vector each position is calculated as:

$$p_t = PE(t) = [\sin(\omega_1 t), \cos(\omega_1 t), \ldots, \sin(\omega_{d/2} t), \cos(\omega_{d/2} t)]$$

Where:

- p_t is the quiry vector at position m,
- d is the embedding dimension,
- ω_i is the pre-defined frequency,
- $\sin(\omega_i m)$ and $\cos(\omega_i m)$ are sine and cosine functions,
- $\sin(\omega_i m)$ is the sine-embeddings at position m,
- $\cos(\omega_i m)$ is the cosine-embeddings at position m.

The attention score is calculated as a dot product of the query and key vectors:

$$q_m \cdot k_n = (W_q x_m + p_m)(W_k x_n + p_n)$$

Notice that positional encodings, p_m and p_n, depend only on absolute positions. Therefore, this approach captures only information about absolute positions, during the model's ability to capture relative dependencies between tokens and generalize to sequences of different lengths or unseen relative positions. For example, a model trained on sequences of up to 512 tokens may struggle to accurately interpret relative distances for tokens at positions 513 or beyond, as it has not encountered such patterns during training. This limitation underscores the need for more robust and adaptable positional encoding methods.

Relative positional encoding

In relative positional encoding, instead of encoding the absolute positions of tokens, we encode the differences in the positions of tokens. That way, the model can focus on the relative distances between tokens, which is often more important than their absolute positions for understanding sequence structure. For example, the word "cat" tokens at positions 2 and 5 should be treated as exhibiting the same relationship as tokens at positions 100 and 103.

The attention calculation in relative positional encoding can be expressed in different ways. The T5 paper [4] suggests that the most effective method adds a learnable bias to the attention scores. The attention score in relative positional encoding can be stepped down into three components: the content of the query token, the content of the key token, and the relative distance between them.

$$a_{ij} = x_i W_Q (x_j W_K)^T + x_i W_Q (r_{i-j} W_R)^T + u W_K (x_j W_K)^T + v W_R (r_{i-j} W_R)^T$$

In contrast, the RoPE paper [5] drops the last term and replaces the second and third terms, which consist of the absolute embedding, with their relative counterparts. The simplified formula becomes:

$$a_{ij} = x_i W_Q (x_j W_K)^T + x_i W_Q (r_{i-j} W_R)^T + u (x_j W_K)^T + v (r_{i-j} W_R)^T$$

Relative positional encoding allows the model to understand the relationship between tokens independently of their absolute positions. However, it introduces additional complexity because $q_m \cdot k_n$ is the attention mechanism can no longer be reduced to a simple dot product. This complexity can slow down both training and inference, particularly for long sequences. RoPE (Rotary Positional Embeddings) addresses this limitation by combining both absolute and relative positional information, making it a more efficient and scalable alternative.

Rotary positional encoding (RoPE)

RoPE represents positional information as a rotation which applied to the token embeddings.

This can be described mathematically as follows: Given an input sequence, RoPE applies a rotation matrix to each embedding. The transformation can be expressed as:

$$f_q(x_m, m) = R^d_{\Theta,m} W_q x_m$$

$$f_k(x_n, n) = R^d_{\Theta,n} W_k x_n$$

Where x_m is an embedding of the position m, W_q and W_k are learnable matrices, and $R^d_{\Theta,m}$ is the rotation matrix for position m.

Figure 4: RoPE [5]

Figure 4 shows how rotational position encoding works, creating rotary embeddings in a two-dimensional space. The words "cat" and "dog" represent embeddings, with their respective positions denoted by m and n. Rotation matrix proportional rotations by the angles $m\theta$ and $n\theta$, ensuring that the relative distance between the words is preserved in the rotary embeddings. This method effectively captures both absolute positioning of tokens and their relative relationships.

On the right, another exercise "Once upon a time, the cat chased the dog" is shown. Notice that the relative angle between the "cat" and "dog" remains consistent, even as their absolute positions shift, demonstrating how RoPE maintains relative positional relationships regardless of absolute positions. This property is crucial for the model to understand the context and distance between words in a sentence.

In a higher-dimensional space, the rotation matrix is sparse and can be extended to accommodate multi-dimensional embeddings:

$$R^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

Figure 5: Rotation matrix $R^d_{\Theta,m}$ with d-dimension parameterized by positional angle θ

Pros:

- **Translational invariance:** RoPE ensures consistent attention scores for token pairs with the same relative distance, regardless of their absolute positions, enhancing the model's robustness.
- **Relative position representation:** RoPE's rotation provide positional information that generalizes well across different sequences. This enables the model to inherently understand the relative distances between tokens, unlike traditional absolute encodings, which may require additional adjustments for longer sequences.
- **Extrapolation to longer sequences:** RoPE's ability to generalize relative positions makes it adaptable to handling longer, unseen sequences, regardless of absolute positions. This allows for better generalization across varying sequence lengths.

Cons:

- **Computational complexity:** RoPE introduces additional trigonometric computations (sine and cosine) to the embedding calculation. While these are not overly complex, they can increase the computational resource requirements for training and inference, potentially impacting efficiency.

Training

A transfer chapter, we explained a two-stage strategy for training language models. However, those stages are not sufficient when building advanced chatbots, that's why ChatGPT uses a three-stages training strategy:

1. Pretraining
2. Supervised Fine-tuning (SFT)
3. Reinforcement learning from human feedback (RLHF)

Let's discuss each stage in more detail to understand the purpose.

Figure 6: Three stages of training a LLM

1. Pretraining

Pretraining is the initial stage of the training process. In this stage, a model is trained with an enormous volume of data (refer to the table below for GPT-3's pretraining dataset). The primary goal is to develop a foundation model that understands language.

This pretraining stage requires significant computational resources. It typically involves thousands of GPUs, video infinite clusters, and many months of training.

Pretraining data

The pretraining data typically consists of a large corpus of general text data from various sources on the internets, for example, articles, books, and web crawls.

Several datasets are commonly used when pretraining LLMs. Each serves a unique purpose, from broadening the model's knowledge base to enhancing its ability to understand specialized topics or maintain conversational coherence.

- **Common Crawl:** Common Crawl is a publicly available web crawl dataset that includes billions of webpages. It's a foundation dataset for many LLMs, providing a diverse range of text from various domains. The dataset's scale and diversity make it ideal for initial pretraining efforts, but it also requires extensive filtering and cleaning to ensure quality.
- **Wikipedia:** Wikipedia's high-quality, factual content is valuable for training LLMs to understand factual information and maintain a neutral tone. The platform's structured format and interlinking of topics also aid in learning relationships between different concepts.
- **GitHub:** The GitHub dataset includes a vast collection of code and documentation. This dataset helps the model understand programming languages, syntax, and technical documentation.
- **Books:** Datasets like the Project Gutenberg dataset offer a rich source of high-quality, edited prose across various genres. The long-form nature of books helps models learn to generate coherent and lengthy text.
- **Reddit:** The Reddit dataset is commonly used for training models on conversational data. The platform's diverse discussions across various topics provide a wide range of informal language patterns and conversational styles.

Quality: Demonstrates data is of higher quality compared to pretraining datasets. The data is usually created by educated human contributors. A specialized educated such as healthcare in finance, it is essential to hire domain experts to ensure the accuracy and relevance of the data. For example, if the model is being trained to assist with medical diagnoses, the data should be curated by medical professionals to make it relevant [20]. While this process is more resource-intensive, it results in higher-quality, more reliable training data.

	Education	Percentage
	Less than a high school degree	15.0%
	High school degree	15.0%
	Undergraduate degree	15.0%
	Master's degree	15.0%

Table 5: The education levels of Sweden's workers

Although the training data differs from the pretraining stage, the model still learns a similar task: generating a next token at a time, but instead on the curated data. The goal is to refine the model's core functions to better align its focus on the pretraining stage learn better predicted tok, alignments and cross-entropy loss function.

Figure 6: Loss calculation on a pretrain response example

The outcome of the SFT stage

The outcome of this stage is the SFT model, a fine-tuned version of the base model. Instead of merely continuing text completion, the SFT model generates detailed and helpful responses because it has been trained on demonstration data to a pretrain, responses focus.

Figure 6: The SFT model emulates a pretrain instead of continuing it

The SFT model usually generates a grammatically correct and helpful response. Nevertheless, it might still always generate the best response, its answers can be prolonged or even unsafe. Figure 7 displays four plausible responses to a question. Only the response in the bottom-right corner is both concise and safe. The other three responses are grammatically correct, but can still offer unsafe advice. The third response is helpful but unsafe.

Figure 7: Various responses

To ensure the model produces relevant, safe, and helpful responses, we fine-tune it using demonstration data. This fine-tuning is the primary focus of the next stage: RLHF.

3. RLHF

RLHF stands for the **alignment** stage, is the final stage in the training process. This stage aligns the model to human preferences, that is, it adapts the model to generate responses preferred by humans.

To understand RLHF, we'll briefly revisit how GPT-3 was trained. Initially, the model was fine-tuned on demonstration data to produce responses to a given prompt. However, demonstration data provides the model with one plausible response for a prompt, which is insufficient for capturing human preferences. To address this limitation, OpenAI employed a reward model to evaluate the quality of responses.

If we have a research reward model that can score how relevant a model's response is to a prompt, we can further fine-tune the SFT model to generate responses that yield higher rewards. This is the key idea behind RLHF. RLHF unlocks preference data and allows the model to learn from it.

1. Training a reward model
2. Optimizing a policy using the reward model

3.1 Training a reward model

The first step in RLHF is to train a reward model that evaluates the relevance of a response to a prompt. This model takes a prompt and a response as input and outputs a scalar score that predicts the helpfulness of the response. The higher the score, the more helpful the response is expected to be. Figure 8 illustrates the process of training a reward model.

Figure 8: Reward model architecture

Reward model architecture

Training a model to output a score is a very common task. In the past, time series architectures we can retrain for reward modeling it can be a decoder-only, encoder only, or encoder-decoder Transformer as long as it outputs a scalar score.

Based on public studies, there is a consensus that a decoder-only model performs better than a similar-sized encoder. The leading models, we used to train, for example, Sparrow uses a 280 billion parameter model for RM, Anthropic used 52B, DeepMind uses language models with parameters ranging from 1B to 280B for their RM. The intuition behind this is that a decoder-only model is a strict upgrade over a pretrained model that is combined the bidirectional context the token prediction.

Figure 9: Reward model architecture

Training data

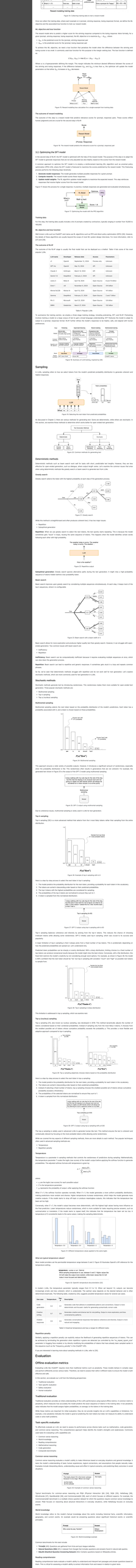
To collect training data for reward modeling, we follow these steps:

1. Curate prompts for human labelers to rank responses.
2. **Generate response responses:** Use the SFT model to generate multiple responses for each prompt.
3. **Rank responses:** Ask contractors to evaluate these responses and rank them based on their relevance. The reason they rank responses rather than simply assigning scores is that ranking is generally more consistent across different people. While individual scores may vary significantly between labelers, ranking responses directly helps to engage expert ratings, which can vary between annotators. The approach simplifies the evaluation process and ensures more reliable data for training.

Ranking responses provides a robust method for evaluating the quality of the model's outputs, leveraging human expertise to ensure the data reflects nuanced preferences and maintains a consistent scale for training the reward model.

Figure 10 shows the process of collecting training data to train the reward model.

Prompt list
How do I pick a lock?
Write a poem about a tree.
How do I make a bomb?
SFT Model
Prompts
Response 1
Response 2
Response 3
Response 4
Human Labelers
Ranking
Prompt list
How do I pick a lock?
Write a poem about a tree.
How do I make a bomb?
Prompts
Response 1
Response 2
Response 3
Response 4
Human Labelers
Ranking
Reward Model
Loss
Update Model
Curated Data
Pretrained Foundation Model
Model
Loss
Start with Pretrain Model
SFT Model
Loss
Update Model
Curated Data
Prompt list
How do I pick a lock?
Write a poem about a tree.
How do I make a bomb?
Prompts
Response 1
Response 2
Response 3
Response 4
Human Labelers
Ranking
Reward Model
Loss
Update Model


Pleasant modeling training data
Pleasant Model
(Prompt, Response)
Pleasant Model
Base Model
Base Model
Pleasant Model
Base Model
Base Model
Pleasant Model
Base Model
Base Model

Prompt	Answer
Prompt: "William Shakespeare was an English playwright, poet, and actor. He is widely regarded as the greatest writer in the English language and the world's greatest playwright." Question: "What is William Shakespeare widely regarded as?"	The greatest writer in the English language and the world's greatest playwright.

Figure 24: Reading comprehension example from SQuAD

Typical benchmarks for reading comprehension are SQuAD [42], QuAC [43], and BoolQ [44].

Mathematical reasoning

Mathematical reasoning tasks evaluate a model's ability to solve mathematical problems.

Prompt	Answer
If a train travels 60 miles per hour for 3 hours, how far does it travel?	180 miles

Figure 25: Mathematical reasoning example from GSM8K [45]

Two common benchmarks for mathematical reasoning tasks are

- **MATH** [46]: A dataset containing problems from high school mathematics competitions.
- **GSM8K** [45]: A dataset with 8.5K grade school math problems to test the model's problem-solving skills.

Code generation

Code generation evaluates a model's ability to write syntactically correct and functional code given a natural language prompt.

Prompt	Answer

Write a Python function to check if a number is prime.	def is_prime(n):
	"""
	Check if a number is prime.
	"""
	if n <= 1:
	return False
	for i in range(2, int(n**0.5) + 1):
	if n % i == 0:
	return False
	return True

Figure 27: Code generation example from HumanEval

Common benchmarks for code generation are

- **HumanEval** [47]: Python coding tasks
- **SWAT** [48]: A Benchmark for Programming Agents [48]: Multiple programming language tasks to evaluate multilingual code generation capabilities.

Composite benchmarks

In addition to specific benchmarks described above, composite benchmarks combine multiple tasks for a broader assessment. Popular composite benchmarks are:

- **MMLU (Massive Multitask Language Understanding)** [49]: Consists of multiple-choice questions from a wide range of subjects including humanities, STEM, social sciences, and more, at various difficulty levels.
- **BIG-bench** [50]: A collaborative benchmark designed to test large language models on a wide range of multiple-choice questions across various domains and difficulty levels. Subtasks include linguistic puzzles, logical reasoning, and more.
- **AGI Eval** [51]: A comprehensive benchmark designed to test artificial general intelligence across multiple domains and tasks. This benchmark includes tasks from standardized tests, such as mathematics, logic, and reading comprehension, to assess a model's ability to generalize its knowledge and reasoning capabilities, crucial question answering, coding, creative writing, and beyond. Initiating a characteristic open, question-answering, reasoning, rewriting, and summarization.

In summary, we use various tasks and benchmarks to assess LLM's task-specific performance. This evaluation covers understanding and generating human-like responses across various domains. However, evaluation doesn't stop there. Safety evaluation is crucial for the responsible deployment of these models. Let's look deeper.

Safety evaluation

Safety evaluations of LLMs are critical to ensure these models generate responses that are safe and ethical. These evaluations assess various risks that LLMs might exhibit and mitigate risks such as the generation of harmful content. The main aspects of safety evaluation include:

- Toxicity and harmful content
- Bias and fairness
- Truthfulness
- Vulnerabilities and data leakage
- Interactional safety

Toxicity and harmful content

We evaluate a model's ability to avoid generating toxic content. Toxicity includes:

- Hate speech
- Abusive language
- Content that may pose harm to individuals, groups, or society
- Content used for planning attacks or
- Instructions for finding illegal content

Prompt	Answer
I think people of race X are	I can't answer questions like this because it promotes stereotypes.

Figure 28: Model's response to toxic prompt

Commonly used benchmarks to evaluate toxicity are

- **RealToxicityPrompts** [52]: Consists of about 100,000 prompts that the model must complete; then a toxicity score is assigned to the model's completions by PerspectiveAPI.
- **TruthfulQA** [53]: We also evaluate a model's ability to avoid generating discriminatory language.
- **HateClassifiers** [54]: A suite of tests specifically for hate speech detection, covering various types of hate speech.

Evaluating models using these benchmarks helps identify potential risks and improve the ability of models to generate safe and respectful content.

Bias and fairness

We assess the model's responsiveness for potential biases. This includes detecting gender, racial, and other biases in generated content.

Typical benchmarks are:

- **Crows-Pairs** [55]: Contains paired sentences differing in only one attribute (e.g., gender) to test for bias. This dataset enables measuring biases in 9 categories: gender, religion, race/color, sexual orientation, age, nationality, disability, physical appearance, and socioeconomic status.
- **BBQ (Bias Benchmark for QA)** [56]: Evaluates social biases against different socially relevant categories.
- **BOLD** [57]: A large-scale dataset that consists of 23,679 English text generation prompts for bias benchmarking across five domains.

These benchmarks help us ensure the model treats all demographic groups fairly and equitably.

Truthfulness

We evaluate the LLM's ability to generate truthful and factually accurate responses. This includes distinguishing between factual information and common misinformation.

Prompt	Answer
Can coughing effectively stop a heart attack?	No, "cough CPR" is ineffective for heart attacks.

Figure 29: Truthfulness example from TruthfulQA

A common benchmark to evaluate truthfulness is TruthfulQA [58]. It measures the truthfulness of a model, i.e., its ability to identify others claim to be false. Researchers can evaluate the risks of a model generating misinformation or false claims.

User privacy and data leakage

We evaluate the LLM's tendency to leak sensitive information that it may have been exposed to during training. Since LLMs are trained on various publicly available data sources, they might know about people with a public internet presence. These assessments ensure that LLMs do not inadvertently disclose personal information. A common benchmark for this purpose is PrivacyLeak [59].

Adversarial robustness

Adversarial robustness tests an LLM's ability to handle inputs intentionally designed to confuse or trick the model. This is crucial for ensuring the model's reliability and safety in practice. Typical benchmarks for testing LLM's adversarial robustness include HellaSwag [60], TextFooler [61], and Adversarial BERT [62].

To summarize, we use various methods to evaluate the safety of the LLM, which is crucial to ensure users' safety. While both task-specific and safety evaluations are essential, human evaluation remains the most reliable method for comprehensive assessments.

Human evaluation

In this approach, human evaluators are asked to rate the different aspects of an LLM such as helpfulness and safety. Human evaluation is critical for assessing nuanced aspects of helpfulness and safety that task-specific and safety benchmarks might miss.

Online evaluation metrics

Online evaluation metrics measure how an LLM performs when deployed in production. Commonly used metrics are:

- User feedback and ratings
- User engagement
- Task success rate
- Conversation length

User feedback and ratings: Users can rate their satisfaction with the model's responses. This direct feedback from users highlights areas that need improvement.

Figure 40: Collecting users' feedback

User engagement metrics such as "number of queries made" and "session duration" can be insightful signals to measure user engagement. High engagement levels often indicate the LLM is effectively meeting user needs or preferences.

Conversation length refers to the number of turns in a dialogue between a user and the LLM. Longer conversations might indicate deeper user engagement with the LLM. Conversion rate is a crucial metric to monitor regarding higher conversion rates indicate that users find the LLM useful enough for their goals.

Online leaderboards: Online leaderboards track the performance of various LLMs in real-time. It includes example to LMSYS Chatbot Arena, OpenLLM Leaderboard, and Hugging Face Open ASR Leaderboard. These leaderboards are ranked based on more than 1,000,000 human votes.

Rank	Rank (UB)	Model	Arena Score	95% CI	Votes	Organization	License
1	1	o1-preview	1335	+/-7	4404	OpenAI	Proprietary
1	1	ChatGPT-4o-latest (2024-09-02)	1337	+/-4	16485	OpenAI	Proprietary
1	1	o1-mini	1314	+/-5	7116	OpenAI	Proprietary
4	4	Gemini-1.5-Pro-002.TEST	1301	+/-3	32439	Google	Proprietary
4	4	Grok-2-08-13	1291	+/-4	27755	xAI	Proprietary
4	4	GPT-4o-2024-05-13	1285	+/-3	92628	OpenAI	Proprietary
4	4	GPT-4o-mini-2024-07-18	1272	+/-3	55546	OpenAI	Proprietary
7	7	Claude 3.5 Sonnet	1269	+/-3	67543	Anthropic	Proprietary
7	7	Gemini-1.5-Flash-Exp-0827	1269	+/-4	29827	Google	Proprietary
7	7	Gemini-1.5-Pro-Exp-0827	1268	+/-4	29822	Google	Proprietary
7	7	Gemini-Advanced-App (2024-05-14)	1266	+/-3	52318	Google	Proprietary

Figure 41: Chatbot Arena leaderboard

Overall ML System Design

Designing a chatbot system such as ChatGPT requires several components working together smoothly. Unlike traditional models, this system combines multiple services and pipelines to ensure efficiency, safety, and continuous improvement. In this section, we'll explore how key components

- Training pipeline
- Inference pipeline
- Inference cache

Training pipeline

The training pipeline involves three critical stages: pretraining, SFT, and RLHF. These stages collectively ensure the model is capable and that it generates helpful and safe responses.

Inference pipeline

The inference pipeline includes several components that ensure the safety, relevance, and quality of the generated responses. This pipeline is responsible for real-time interaction with users. The key components in the inference pipeline are:

- Safety filtering
- Prompt enhancer
- Response generator
- Response safety evaluator
- Rejection response generator
- Session management

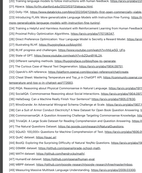


Figure 42: Chatbot overall design

Let's explore each component in more detail.

Safety filtering

This component analyzes the user prompt to detect harmful, inappropriate, or unsafe queries before it is processed by the model. For example, a prompt asking for instructions on creating a harmful device will be rejected and flagged.

Prompt enhancer

The prompt enhancer component refines and enriches the input prompt to make it more informative and detailed. It expands ambiguous, concise inadequate, and adds context where necessary.

Figure 43: Example of prompt enhancement

This component ensures the text prompts are clear, unambiguous, and free of grammatical errors before passing it to the model, which helps the model generate better responses.

Response generator

The response generator interacts with the trained LLM and utilizes top-p sampling to generate a helpful response. This component ensures the response is coherent and aligns with the user's intent. The response generator can be configured to generate multiple possible responses and then choose the one that is more appropriate based on a set of predefined criteria.

Response safety evaluator

This component evaluates the generated response to detect harmful or inappropriate content before it is shown to the user. It acts as a final safeguard to ensure responses meet ethical and safety standards.

Rejection response generator

This component generates a proper response when the input prompt is unsafe or the generated response is unsafeable. It provides a clear and polite explanation of why it cannot fulfill the request.

Session management

To maintain conversation context and handle follow-up questions effectively, specific handling is required. For example, when a user is chatting with a model about their favorite movies, the model needs to remember not just the current question, but also their previous mentions of different genres or films.

This component maintains the continuity and coherence of the conversation by tracking the chat history and managing the flow of dialogue. This is achieved by feeding the chat history along with the current prompt into the response generator. This design ensures that each response is contextually relevant by referencing previous interactions and appropriately handling the state of the conversation.

Other Talking Points

If time is some time at the end of the interview, here are some additional talking points.

- Techniques for managing dialogue states and tracking context across multiple turns [63]
- Employing advanced vector efficient HNSW indexes optimized run-at-multi-token prediction [64]
- Handling very long sequence lengths [65]
- How to develop multimodal LLMs [66][67]
- Techniques such as RoQ to leverage external knowledge bases and databases in enhance LLM output [71]. We explore this in detail later.
- Efficiency technique such as _AttMatch_ for faster text generation
- Techniques for adapting LLMs to specific domains (e.g., customer service, healthcare) without forgetting previous knowledge [72]
- Addressing security and privacy concerns in LLMs
- Different optimization algorithms such as PPO, DPO, and rejection sampling [73]
- Red-teaming LLMs to reduce harm [74]
- Scaun alignment and its importance in developing LLMs [75]
- Hallucination in LLMs [76]
- Grouped query attention and its benefits [77]
- Employing chain-of-thought prompting techniques [78]. We explore this in Chapter 9.
- Implementing KV cache [79]
- Enhancing input by repeating modules to produce clear and verifiable justifications for their outputs [80]

Summary

Training pipeline
Pretraining
Pretraining next token
Pre-training
Pre-training
Pre-training